

تقرير مشروع متجر الكتب مع تطبيق مفاهيم الهندسة المتوازية

السنة الرابعة : كلية الهندسة المعلوماتية - الفريق 15

محمد ملاذ مزيد احمد	محمد عادل المحمد	ورد عدنان اختيار
محمد احمد ياسر صيرفي	المهندس : حذيفة عقيل	محمد غيث محمد جمال شبقجي

مقدمة المشروع ومنهجية العمل

المشكلة التي يعالجها المشروع ليست إنشاء متجر إلكتروني تقليدي فقط بل بناء محرك خلفي يبقى مستقرا عندما تصل طلبات كثيرة في وقت متقارب. في هذه الحالة تصبح صحة البيانات أهم من سرعة الاستجابة المنفردة لأن بيع كمية أكبر من المخزون أو إنشاء طلب ناقص أخطر من زيادة بسيطة في زمن التنفيذ.

اتبعت عملية التطوير منهجا تدريجيا. بدأ العمل بتثبيت الوظائف الأساسية ثم تحديد نقاط التزامن الحساسة ثم إضافة المعاملات والأقفال والطوابير والذاكرة المؤقتة . بعد ذلك تم تشغيل اختبارات منفصلة لكل ظاهرة بدلا من الاعتماد على اختبار واحد عام. هذا الفصل بين السيناريوهات مهم لأنه يميز بين فشل الأداء وفشل منطق الأعمال وفشل سلامة البيانات.

أهداف التصميم

- منع بيع مخزون غير موجود عند تنفيذ طلبات متزامنة.
- حصر عدد الأعمال المتوازية بما يناسب قاعدة البيانات والذاكرة.
- تقليل زمن الطلبات التي لا تحتاج انتظار التنفيذ الكامل.
- توزيع الحمل على أكثر من نسخة من التطبيق.
- تقليل الاستعلامات التجميعية المكلفة .
- توفير قياسات قابلة للتكرار قبل التحسين وبعده.

المتطلبات :

رقم	المتطلب	التطبيق	دليل التحقق
1	حماية البيانات المشتركة	قفل المنتج والسلة ومعاملة checkout	اختبار 100 مستخدم على مخزون 50
2	إدارة الموارد	DB pool وطابور بحد انتظار وعمال concurrency محدد	اختبار ضغط السلة ومراقبة الموارد
3	المعالجة غير المتزامنة	Redis BullMQ وإرجاع HTTP 202	5000 عملية إضافة إلى السلة
4	Batch Processing	مهمة يومية تقرأ البيانات في chunks	SalesBatchService بحجم افتراضي 500
5	توزيع الأحمال	least_conn و Nginx و NestJS	مقارنة tail latency قبل وبعد
6	Distributed Caching	Redis مشترك مع TTL و invalidation	best- Benchmark لمنتجات sellers
7	التحكم في الأقفال	Pessimistic lock مع ترتيب productId	عدم وجود مخزون سالب أو overselling
8	ACID	معاملة واحدة للمخزون والدفع والطلب والسلة	مقاطعة العملية و نجاح كامل أو rollback
9	Stress Testing	JMeter مع آلاف المستخدمين الافتراضيين	28000 عينة و 0% أخطاء
10	Benchmarking	قياس قبل وبعد ومؤشرات percentiles	تحسن متوسط 91.63% مع Redis

الأدوات والتقنيات :

الأداة	سبب اختيارنا لها	المقايضة أو القيد
+ NestJS TypeScript	هيكل Modules و Controllers و Services و Guards و Interceptors واضح ومناسب للتوثيق	عملية Node واحدة تعتمد على event loop وليست توازيا حسابيا حقيقيا لذلك تم استخدام عدة نسخ و Workers
PostgreSQL	معاملات ACID وأقفال صفوف وفهارس قوية	الأقفال الطويلة تقلل التوازي لذلك يجب أن تبقى المعاملة قصيرة
TypeORM	يوحد التعامل مع الكيانات والمعاملات والأقفال	synchronize مناسب للتجربة فقط والمهاجرات أفضل للإنتاج

Redis	ذاكرة مشتركة سريعة ودعم BullMQ	الكاش والطاير في نفس الخادم يشارك في الذاكرة ونقطة الفشل
BullMQ	طوابير موثوقة مع retry concurrency	HTTP 202 لا يضمن نجاح المهمة لاحقا لذلك يلزم تتبع job status
Nginx	خفيف ومستقر ويقدم reverse proxy load balancing	لا يحل وحده مشكلة الحالة المشتركة لذلك بقيت Redis و PostgreSQL مشتركتين
Docker Compose	تشغيل قابل للتكرار لست خدمات	ليس بديلا عن orchestration إنتاجي مثل Kubernetes
JMeter	يدعم Timers و Threads و Percentiles	تشغيله من نفس الجهاز ومن الواجهة قد يؤثر في النتائج العالية
Postman	فحص يدوي للواجهات وموارد النسخة	ليس أداة قياس ضغط

ملاحظة : هناك صور اختبارات كثيرة في المجلد غير مضمنة بالتقرير

المحور	الغرض	الأدوات
الطلب الثاني	ضبط استهلاك الموارد تحت الضغط ومنع انهيار التطبيق عند تراكم الطلبات	TypeORM, Redis, BullMQ, NestJS
الطلب الخامس	توزيع الطلبات على أكثر من نسخة من التطبيق بدل الاعتماد على نسخة واحدة	Nginx, Docker Compose, خوارزمية least_conn
اختبار الأداء	قياس زمن الاستجابة ونسبة الأخطاء ومعدل الطلبات تحت الضغط	Aggregate تقارير , Apache JMeter Summary و

المكون	الدور في النظام
NestJS	تنظيم التطبيق إلى وحدات، وتوفير الحراس، والحقن الاعتمادي، و ربط المتحكمات بالخدمات

تخزين البيانات وتنفيذ المعاملات وقفل السجلات عند الحاجة لمنع التعارض	TypeORM مع PostgreSQL
تخزين المهام المؤجلة وتشغيلها خارج دورة الطلب المباشر	BullMQ و Redis
استقبال الطلبات وتوزيعها على نسخ التطبيق المختلفة	Nginx
تشغيل قاعدة البيانات و Redis ونسخ التطبيق و Nginx ضمن بيئة واحدة	Docker Compose
تنفيذ اختبارات الضغط وقراءة المؤشرات الإحصائية للأداء	JMeter

الحلول المستخدمة في الطلب الأول

المتطلب الأول حماية البيانات من التضرر

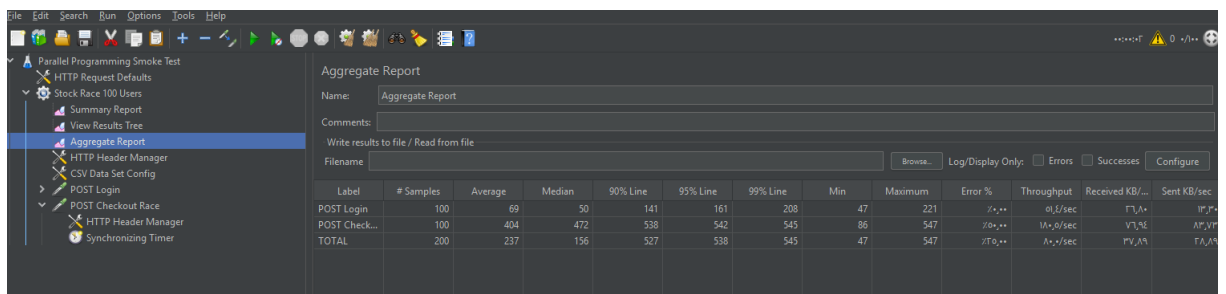
المورد الأكثر حساسية هو عدد القطع في Product. المشكلة التقليدية تحدث عندما يقرأ طلبان القيمة نفسها ثم يخصم كل منهما منها. إذا كانت الكمية 1 فقد ينجح الطلبان ويصبح المخزون غير صحيح. لذلك لا يكفي التحقق من الكمية قبل بدء المعاملة.

الحل المستخدم

يتم فتح معاملة ثم قراءة المنتج بقفل pessimistic_write. الطلب الثاني ينتظر تحرير القفل ثم يقرأ القيمة الجديدة. بعد ذلك يتم فحص الكمية وتحديثها. كما تجمع كميات المنتج المكرر وترتب المنتجات حسب productId قبل أخذ الأقفال. ترتيب الأقفال ثابت ويقلل احتمال deadlock عندما يحتوي الطلب على عدة منتجات.

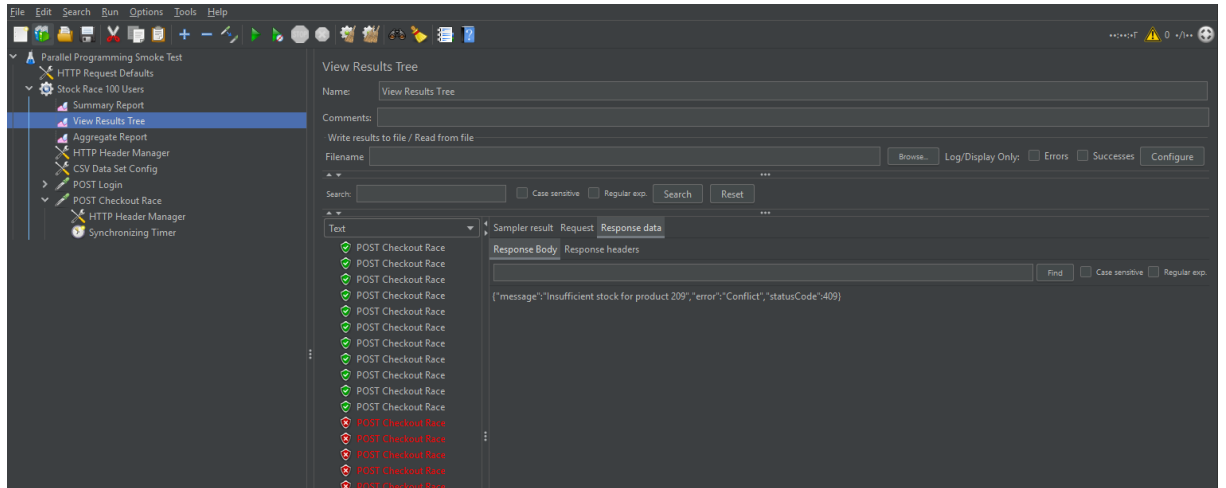
الدليل التجريبي

تم تجهيز 100 مستخدم و 100 سلة تحتوي كل منها قطعة واحدة من المنتج نفسه. كانت كمية المنتج 50. استخدم Synchronizing Timer حتى تبدأ طلبات checkout المئة معا تقريبا. التوقع الحسابي هو نجاح $\min(100, 50) = 50$ طلبا ورفض 50=50-100 طلبا.

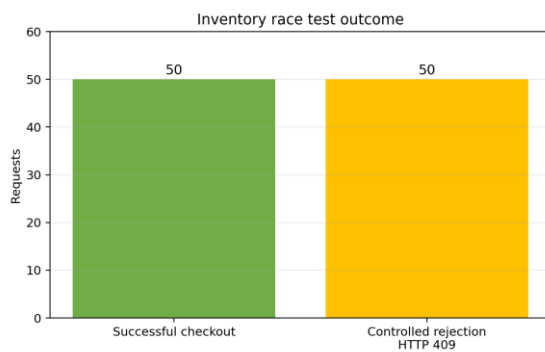


Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
POST Login	100	69	50	141	161	208	47	221	0.00	0.0/sec	0.0	0.0
POST Check...	100	404	472	538	542	545	86	547	0.00	0.0/sec	0.0	0.0
TOTAL	200	237	156	527	538	545	47	547	0.00	0.0/sec	0.0	0.0

الشكل 2: ملخص اختبار سباق المخزون



الشكل 3: الرفض المنضبط بعد نفاذ المخزون برمز HTTP 409



توزع النتائج المتوقع والمتحقق

قيم الزمن والنسب في JMeter

الحلول المستخدمة في الطلب الثاني : إدارة الموارد الحاسوبية

السبب	مكان التطبيق	الحل
تحديد عدد الطلبات المسموح بها خلال فترة زمنية، وإرجاع الحالة 429 عند تجاوز الحد	src/app.module.ts	Rate Limiting
فحص عدد المهام المنتظرة قبل قبول مهمة جديدة في السلة، وإرجاع الحالة 503 عند امتلاء الصف	src/carts/carts.controller.ts	Queue Backpressure
تحديد عدد المهام التي يمكن للمعالج تنفيذها بالتوازي حتى لا يضغط على قاعدة البيانات و Redis	src/carts/carts.processor.ts	Worker Concurrency
تحديد عدد اتصالات قاعدة البيانات بدل تركها غير مضبوطة أثناء الضغط	src/app.module.ts	DB Connection Pool
وضع حد لذاكرة Redis مع سياسة عدم حذف المهام تلقائياً، حتى لا تضيق المهام عند الضغط	docker-compose.yml	Redis Memory Limit

أفضل طلب لاختبار هذا الجزء هو طلب **إضافة منتج إلى السلة**، لأنه يضيف مهمة إلى صف السلة ويؤثر على قاعدة البيانات عند معالجة المهمة.

POST /carts/add

- > ThrottlerGuard
- > ensureCartQueueCapacity()
- > BullMQ cart queue
- > CartConsumer
- > CartsService
- > PostgreSQL transaction

الضابط	القيمة	الهدف
DB_POOL_MAX	10 لكل نسخة	منع فتح اتصالات غير محدودة
Cart worker concurrency	8 لكل نسخة	تنفيذ عدد مضبوط من مهام السلة
CART_QUEUE_MAX_WAITING	5000	رفض الحمل الزائد برمز 503 بدلا من استنزاف الذاكرة
Throttler	200 طلب خلال 60 ثانية افتراضيا	حماية الواجهة في التشغيل العادي
Load-test override	1000000	إزالة تأثير rate limit أثناء القياس فقط

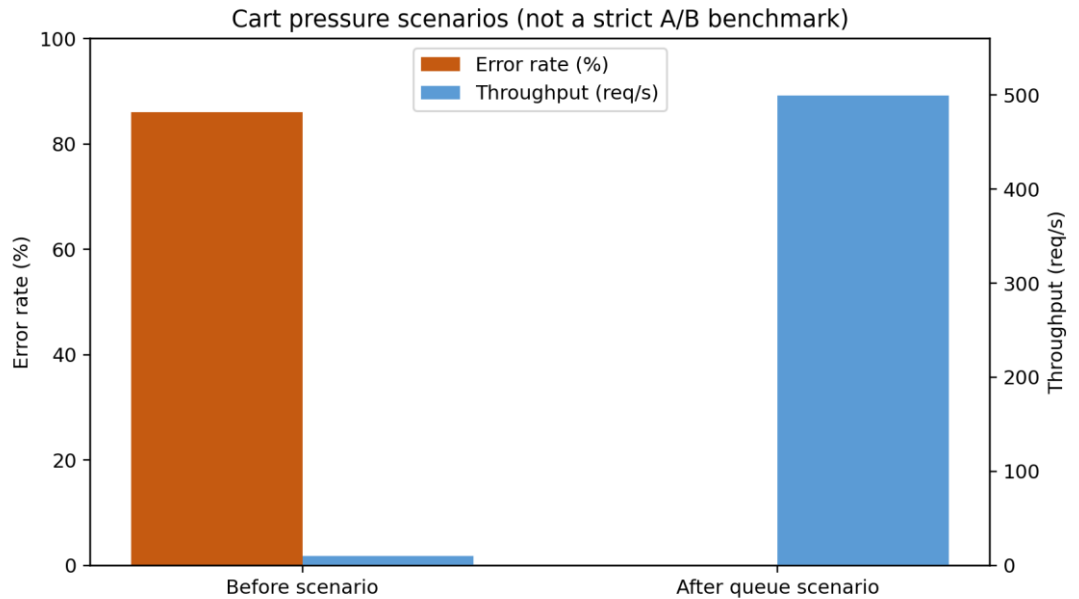
قراءة اختبار السلة قبل وبعد

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Max	Min	MaxError	Error %	Throughput	Success Rate	Start Time	End Time
Agg. Test	100	12.12	12.12	15.15	20.10	25.15	25.15	0.00	0.00	0.00%	4.00	100%	10:00	10:01
Total	100	12.12	12.12	15.15	20.10	25.15	25.15	0.00	0.00	0.00%	4.00	100%	10:00	10:01

سيناريو سابق: 100 طلب و 86% أخطاء

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Max	Min	MaxError	Error %	Throughput	Success Rate	Start Time	End Time
Agg. Test	5000	12.12	12.12	15.15	20.10	25.15	25.15	0.00	0.00	0.00%	4.00	100%	10:00	10:01
Total	5000	12.12	12.12	15.15	20.10	25.15	25.15	0.00	0.00	0.00%	4.00	100%	10:00	10:01

سيناريو الطابور: 5000 طلب و 0% أخطاء



الشكل 4: مقارنة تشغيلية وليست Benchmark متطابق الشروط

الحلول المستخدمة في الثالث: المعالجة غير المتزامنة

الهدف :

- يهدف هذا المتطلب إلى فصل "مسار الطلب الرئيسي (Request-Response Cycle)" عن المهام الثانوية الثقيلة التي لا يحتاج المستخدم للانتظار نتائجها فوراً (مثل إرسال رسائل التأكيد، أو الإشعارات، أو توليد الفواتير). إجبار المستخدم على انتظار هذه العمليات يؤدي إلى زيادة زمن الاستجابة وتجربة مستخدم سيئة.

تطبيقه في المشروع:

- قمت بتطبيق هذا الفصل باحترافية عن طريق دمج BullMQ المدعوم ب Redis فعندما يقوم المستخدم بطلب إضافة إلى السلة أو إتمام الدفع، يستقبل السيرفر الطلب ويلقيه فوراً كـ "مهمة خلفية (Job)" داخل Redis، ويرد فوراً على المستخدم برمز الاستجابة Accepted 202 ليتصفح الموقع بحرية، بينما يتولى BullMQ معالجة تلك المهام الثقيلة في الخلفية بشكل منفصل تماماً عن مسار خط النظام الرئيسي.

الحلول المستخدمة في الطلب الرابع : معالجة البيانات الضخمة على دفعات (Batch Processing)

الهدف :

- عند التعامل مع كميات ضخمة من البيانات (مثل جرد المبيعات اليومية أو تحديث مخزون آلاف المنتجات دفعة واحدة)، فإن معالجة كل سجل على حدة تستهلك اتصالات شبكة كثيفة ومجهددة لقاعدة البيانات. يتطلب هذا البند تجميع هذه البيانات ومعالجتها على شكل مجموعات أو دفعات مقننة ومقسمة (Chunks / Patches) لتقليل العبء وتسريع الأداء الإجمالي.

تطبيقه في المشروع :

- تم تطبيق ذلك برمجيا من خلال آليات التجزئة في BullMQ حيث يتم تقسيم العمليات الضخمة القادمة إلى دفعات برمجية (Patches) ، ويقوم ال Worker بمعالجة كل دفعة بشكل مركز ومتواز، مما يحافظ على توازن استهلاك الذاكرة ويمنع تجميد حلقة الأحداث (Event Loop) الخاصة بـ NestJS أثناء معالجة البيانات الضخمة.

الطلب الخامس: توزيع الحمل

تم تنفيذ الطلب الخامس بتشغيل تلت instances من التطبيق و وضع موزع طلبات أمامها لتمرير كل طلب إلى النسخة المناسبة. تم تعريف النسخ داخل ملف إعداد الحاويات، ثم تم تمرير الطلبات إليها من خلال إعدادات موزع الطلبات . و استخدامنا خوارزمية less conn تختار هذه الخوارزمية النسخة التي لديها أقل عدد من الاتصالات النشطة حالياً. لأن الطلبات ليست كلها بنفس الزمن؛ بعض الطلبات خفيف وبعضها يتعامل مع قاعدة البيانات . هذا ال proxy يلي هو NGINX

```
upstream nest_backend {
    least_conn;
    server api1:3000;
    server api2:3000;
    server api3:3000;
}
```

لماذا لم نستخدم Round Robin

التوزيع بالدور يرسل الطلبات بالتناوب بغض النظر عن انشغال النسخة الحالية. أما خوارزمية أقل الاتصالات فتراعي عدد الاتصالات النشطة لذلك تعطي فرصة أفضل للنسخة الأقل انشغالا في مشروع يحتوي على معاملات وقاعدة بيانات هذا الاختيار أكثر مرونة .

الاختبار

العنصر	القيمة
الطلب المختبر	GET /products
عدد العينات	16000 طلب
نوع التقارير	تقرير تجميعي وتقرير ملخص
هدف القراءة	قياس أثر الضغط العالي على طلب قراءة قبل و بعد الاعتماد على ال load balancing .

القيم من الصور المرفقة مباشرة. تظهر النتائج أن الضغط كان كبيراً على السيناريو المختبر، وأن نسبة الأخطاء مرتفعة ، وهذا يدعم الحاجة إلى حلول الطلب الثاني والخامس .

المؤشر	القيمة	التفسير
عدد العينات	16000	عدد الطلبات التي تم إرسالها إلى طلب قراءة المنتجات
متوسط الزمن	3054ms	متوسط الاستجابة يقارب 3.05 ثانية، وهو مرتفع لطلب قراءة
الوسيط	3674ms	نصف الطلبات تقريباً احتاجت إلى 3.67 ثانية أو أقل
خط 90%	5371ms	تسعون بالمئة من الطلبات كانت أقل من 5.37 ثانية
خط 95%	5522ms	خمسة وتسعون بالمئة من الطلبات كانت أقل من 5.52 ثانية
خط 99%	5951ms	تسعة وتسعون بالمئة من الطلبات كانت أقل من 5.95 ثانية
أعلى زمن	6446ms	أطول استجابة وصلت إلى نحو 6.44 ثانية
نسبة الأخطاء	58.51%	نسبة أخطاء عالية جداً وتدل أن الضغط تجاوز قدرة السيناريو المختبر
معدل التنفيذ	114.1sec	النظام عالج أو سجل نحو 114 طلباً في الثانية لكن مع نسبة أخطاء عالية

Aggregate Report

Name:

Aggregate Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

☐ Errors
 ☐ Successes

Configure

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
Get Products	16000	3054	3674	5371	5522	5951	0	6446	0.0%	111.1/sec	111.1/sec	111.1/sec
TOTAL	16000	3054	3674	5371	5522	5951	0	6446	0.0%	111.1/sec	111.1/sec	111.1/sec

المؤشر	القيمة	التفسير
عدد العينات	20000	عدد الطلبات التي تم إرسالها إلى طلب قراءة المنتجات
متوسط الزمن	410ms	زمن أقل
أعلى زمن	6446ms	نفس أعلى زمن ظاهر في التقرير السابق
نسبة الأخطاء	0%	لم يفشل شيء

معدل التنفيذ	sec/172.4	ارتفع قليلا , معدل الطلبات في الثانية، ويجب قراءته مع نسبة الأخطاء لا وحده
--------------	-----------	--

Summary Report										
Name: Summary Report										
Comments:										
Write results to file / Read from file										
Filename Browse... Log/Display Only: ☐ Errors ☐ Successes Configure										
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get Products	20000	410	1	2634	492.18	0.00%	478.5/sec	327.58	172.43	701.0
TOTAL	20000	410	1	2634	492.18	0.00%	478.5/sec	327.58	172.43	701.0

مقارنة عامة بين ما قبل التحسين وما بعده

النقطة	قبل التحسين	بعد التحسين
قبول الطلبات	قد يقبل النظام عدداً كبيراً من الطلبات حتى تظهر أخطاء اتصال أو ضغط على قاعدة البيانات	يتم رفض الضغط الزائد بشكل واضح عبر 429 أو 503 بدلاً من الانهيار
عمليات السلة	الطلبات قد تتراكم دون فحص واضح لسعة الصف	يتم فحص سعة صف السلة قبل إدخال مهمة جديدة
قاعدة البيانات	احتمال ضغط أعلى بسبب عدم ضبط الاتصالات بالشكل المطلوب	يوجد حد للاتصالات قاعدة البيانات لتقليل الاختناق
توزيع الطلبات	الضغط قد يتركز على نسخة واحدة أو توزيع غير واضح	توجد ثلاث نسخ خلف Nginx ويتم توزيع الطلبات بخوارزمية least_conn
قراءة نتائج الاختبار	الأخطاء العالية قد تظهر ك 502 أو انقطاع اتصال عند تجاوز القدرة	الأخطاء المتوقعة تصبح مضبوطة أكثر مثل 429 أو 503 عند تجاوز الحدود
استهلاك الموارد	-	زاد استهلاك الـ cpu +80% إلى +200% وقت الضغط

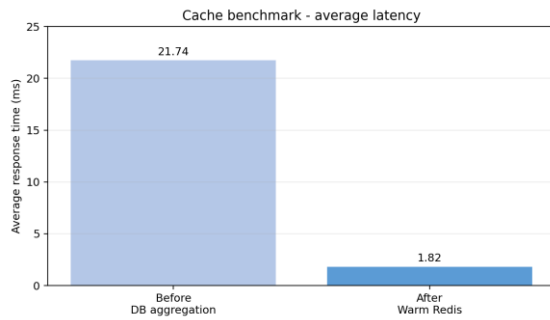
زاد استهلاك ال ram +200% تقريباً عند تشغيل الثلاث كوتينرات

المتطلب السادس : التخزين المؤقت الموزع

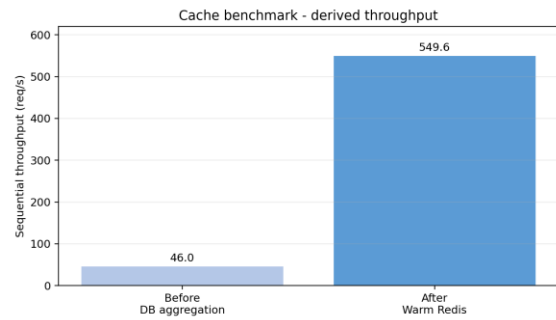
مسار products/best-sellers ينفذ join بين المنتجات وعناصر السلة والسلال والطلبات ثم group by و sum و sort. هذه عملية مناسبة للكاش لأن النتيجة تقرأ كثيراً وتتغير أبطأ من معدل القراءة. تستخدم جميع نسخ التطبيق Redis نفسه لذلك لا تختلف النتيجة باختلاف النسخة.

استراتيجية Cache-Aside :

1. يحاول التطبيق قراءة المفتاح من Redis.
2. عند hit يعيد القيمة مباشرة.
3. عند miss ينفذ استعلام PostgreSQL.
4. يحفظ الناتج JSON مع TTL.
5. عند تعديل بيانات مرتبطة يحذف المفاتيح المتأثرة.



متوسط زمن الاستجابة قبل Redis وبعده



المعدل التسلسلي المحسوب من مدد الطلبات

المرحلة	Average	P50	P95	Throughput
Before - cache cleared	ms 21.74	ms 20.8	ms 30.75	req/s 46
After - warm Redis	ms 1.82	ms 1.8	ms 2.44	req/s 549.6

حساب التحسن: $(21.74 - 1.82) \div 21.74 \times 100 = 91.63\%$. ونسبة معدل التنفيذ التسلسلي $46 \div 549.6 = 11.95$ مرة. هذا throughput مشتق من مجموع مدد طلبات تسلسلية وليس نتيجة اختبار مستخدمين متزامنين لذلك يستخدم للمقارنة داخل السيناريو نفسه فقط.

المتطلب السابع : التحكم في الأقفال

استخدم المشروع نوعين متكاملين من التحكم في التزامن. القفل التشاؤمي يستخدم على السجلات الحساسة التي لا يمكن قبول تعارضها مثل المنتج والسلة والطلب. التحكم المتفائل يستخدم عبر version عند تحديث السلة وعناصرها.

السبب	الآلية	المورد
منع الخصم المتزامن من القيمة القديمة	pessimistic_write	Product
منع تعديل السلة أثناء checkout أو تعديل آخر	pessimistic_write	Open Cart
منع انتقالين متعارضين لحالة الطلب	pessimistic_write	Confirm
رفض التحديث إذا تغيرت السلة بعد قراءتها	Optimistic check	Cart version
رفض تعديل كمية عنصر تغير من طلب آخر	Optimistic check	UserProduct version

أثر الأقفال في الأداء

القفل الصحيح يضحى بجزء من التوازي من أجل الصحة. في اختبار السباق ارتفع وسيط checkout إلى 472 ميلي ثانية لأن الطلبات انتظرت القفل بالتتابع. هذا سلوك متوقع وليس خلافاً. الهدف هو أن الانتظار ينتهي برفض صحيح بدلاً من بيع زائد.

المتطلب الثامن : سلامة المعاملات

تنفذ عملية checkout داخل DataSource.transaction. تشمل العملية قراءة السلة وقفلها والتحقق من العناصر وقفل المنتجات وتخفيض المخزون واعتماد الدفع التجريبي وإنشاء Confirm وربط السلة به. إذا رمى أي جزء استثناء تتراجع PostgreSQL عن كل التغييرات.

ترتيب العملية :

6. قفل السلة المفتوحة للمستخدم.
7. رفض السلة الفارغة.
8. ترتيب المنتجات وقفلها ثم فحص المخزون.
9. تخفيض الكمية.
10. اعتماد payment token التجريبي.
11. إنشاء الطلب بحالة pending.
12. ربط السلة بالطلب مع فحص version.
13. Commit أو Rollback تلقائي.

حد المعاملة الحالية :

خدمة الدفع الحالية محلية وتولد UUID ولذلك يمكن تضمينها داخل المعاملة. بوابة دفع حقيقية خارجية لا تشارك في معاملة PostgreSQL. عندها نحتاج Saga أو Outbox وعمليات idempotent وتعويض refund لأن ACID لا تمتد تلقائياً إلى خدمة خارجية.

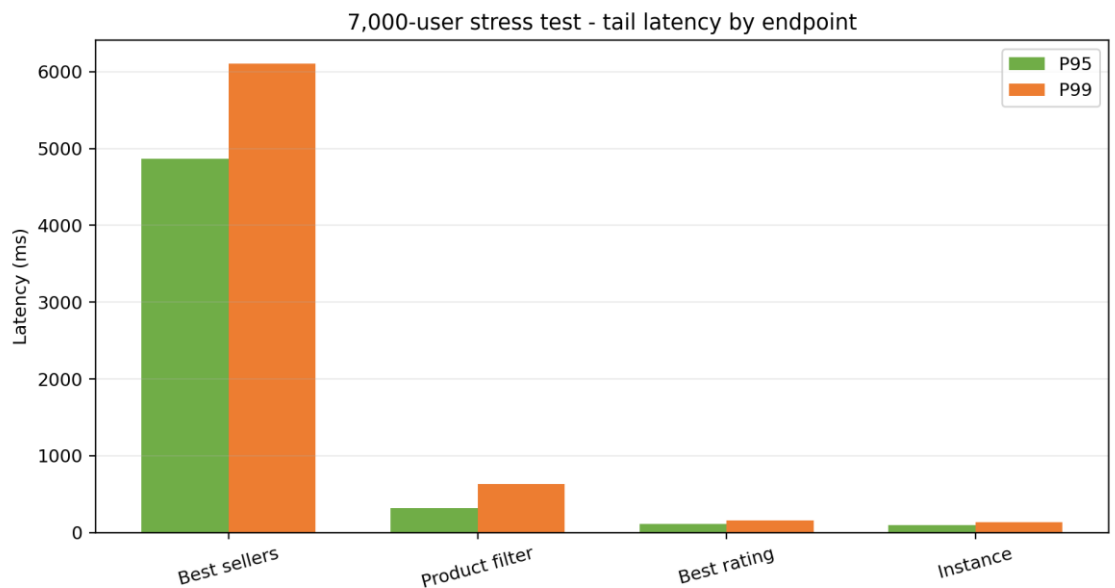
المتطلب التاسع : اختبار الاستقرار تحت الضغط

تجاوز الاختبار الحد الأدنى المطلوب وهو 100 مستخدم. تم تشغيل 7000 مستخدم افتراضي وكل مستخدم نفذ أربعة طلبات قراءة. العدد الكلي $4 \times 7000 = 28000$ عينة.

Write results to file / Read from file												
Filename										Browse...	Log/Display Only:	Errors
										Successes		
										Configure		
Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
GET Best Sell...	7000	544	74	2062	4869	6104	5	6456	٪٠,٠٠	٥٨٨,٧/sec	٨٩٧,٤٧	١١٣,١١
GET Product...	7000	218	190	289	319	630	5	6030	٪٠,٠٠	٥٨٩,٥/sec	١٤٣,٨٧	١١٥,٧٣
GET Best Rat...	7000	95	79	107	115	156	1	6023	٪٠,٠٠	٥٩٠,٥/sec	١٠١٧,٨٤	١١١,٨٨
GET Instance	7000	91	68	91	98	139	1	6415	٪٠,٠٠	٥٩١,٣/sec	٢١٨,٨٦	١٠٥,١٠
TOTAL	28000	237	80	246	309	5418	1	6456	٪٠,٠٠	٢٣٥٢,٣/sec	٣٥١٩,٦٥	٤٤٣,٥٥

الشكل 6: Aggregate Report للاختبار 28000 طلب

المسار	Samples	Average	Median	P95	P99	Error
GET Best Sellers	7000	544	74	4869	6104	٪0
GET Product Filter	7000	218	190	319	630	٪0
GET Best Rating	7000	95	79	115	156	٪0
GET Instance	7000	91	68	98	139	٪0
TOTAL	28000	237	80	309	5418	٪0



الشكل 7: مقارنة P95 و P99 بين المسارات

أبطأ مسار كان best-sellers وظهر له P95 يساوي 4869ms و P99 يساوي 6104ms رغم أن الوسيط 74ms. هذا الفرق الكبير يعني أن معظم الطلبات سريعة لكن مجموعة صغيرة انتظرت بقوة عند الذروة. لذلك لا يكفي القول إن Average الكلي 237ms جيد دون ذكر tail latency.

المتطلب العاشر : القياس وتحليل الاختناق

تم اختيار best-sellers كاختناق لأنه ينفذ استعلاما تجميعيا متعدد الجداول. تم تثبيت endpoint وعدد التكرارات ثم قياس مرحلتين. في المرحلة الأولى يحذف مفتاح Redis قبل كل طلب لضمان الوصول إلى PostgreSQL. في المرحلة الثانية يسخن الكاش مرة واحدة ثم تقاس الطلبات من Redis.

لماذا 50 تكرار :

عشرون تكرارا تكفي لاكتشاف فرق كبير لكنها حساسة لضوضاء النظام. تم اعتماد 50 تكرارا كحل وسط يعطي P95 ذا معنى أفضل دون تحويل القياس إلى اختبار ضغط طويل. مع 50 عينة يمثل P95 تقريبا العينة رقم $\text{ceil}(0.95 \times 50) = 48$ بعد الترتيب.

الاستنتاج :

الفرق الكبير يؤكد أن الاختناق هو التجميع وقاعدة البيانات وليس Nginx أو Redis. JSON serialization مناسب لهذا النوع من القراءة. لكن جودة القياس لا تعني أن كل endpoints تستفيد بنسبة 91.63% لأن النتيجة خاصة بالمسار والبيانات المستخدمة.

تطبيق AOP لمراقبة الأداء

سجل الأداء يمثل اهتماما مشتركا لا يجب تكراره داخل كل Controller. لذلك سجل PerformanceInterceptor عالميا من خلال APP_INTERCEPTOR. يبدأ المؤقت قبل تنفيذ handler ويستخدم finalize بعد النجاح أو الخطأ لحساب المدة.

الحقل	الفائدة
route و method	معرفة العملية المقاسة
handler و controller	تحديد موقع التنفيذ في الكود
error و statusCode	تمييز النجاح والفشل
durationMs	القياس الأساسي
userId	ربط السلوك بالمستخدم عند توفره
instanceId	معرفة النسخة التي نفذت الطلب
timestamp	ربط الحدث زمنيا

لا يسجل interceptor body أو password أو access token أو payment token. ويمكن تعطيل تسجيل الطلبات السريعة عبر PERFORMANCE_LOG_ALL=false مع الاحتفاظ بالأخطاء والطلبات الأبطأ من PERFORMANCE_WARN_MS. أثناء اختبار الضغط استخدم هذا الخيار لتجنب أن تصبح الكتابة إلى السجل جزءا من الاختناق.

endpoint موارد النسخة :

أضيف GET /resources ليعرض hostname و instanceId وعدد الأنوية واستخدام CPU والذاكرة وذاكرة عملية Node ووقت التشغيل. قياس CPU يعتمد عينتين بينهما 100ms ولذلك هو لقطة تقريبية. داخل Docker قد تختلف قراءة الذاكرة حسب دعم Node لـ cgroups لذلك تبقى docker stats أداة تحقق مهمة.

GET <http://localhost:8080/resources>

عرض نتائج الاختبارات وتحليلها

الاختبار	الحجم	النتيجة	التفسير
Cache benchmark	50 + 50 طلبا	Average 21.74ms إلى 1.82ms	تحسن 91.63%
Load balancing read	4000 قبل و 4000 بعد	6ms إلى P99 19ms	تحسن tail latency
Cart queue	5000	11ms ومتوسط 0% أخطاء	زمن قبول الطابور
Inventory race	checkout 100	HTTP 409 50 نجاح و 50	منع overselling
General stress	28000	0% أخطاء	P99 الكلي 5418ms

THE EEEEND